

Unit-2

Exception Handling and Threading

Threading :- It is a way to run multiple instructions in a program simultaneously.

It can help make programs more efficient and improve user experience.

Thread:- A small part of our program code are known as thread.

It is lightweight as compared to process

Uses of Threading :-

some common uses of threading in Python as follows.

1)Concurrent Execution: Threading enables you to execute multiple tasks concurrently. For example, you can use threads to handle network requests, perform time-consuming I/O operations, or carry out computations in parallel, making your program more efficient.

2)GUI Applications: Threading is often used in graphical user interface (GUI) applications to keep the user interface responsive while performing other tasks in the background. By running time-consuming operations in separate threads, you can prevent the GUI from freezing or becoming unresponsive.

3)Network Operations: Threading is useful for handling network-related tasks such as making HTTP requests, downloading files, or running a server. By using threads, you can initiate multiple network operations simultaneously and process their results concurrently.

4)Parallel Processing: Threading allows you to perform parallel processing, especially on multi-core or multi-processor systems. You can split a large computation-intensive task into smaller parts and execute them concurrently in separate threads to take advantage of the available computing resources.

5)Asynchronous Operations: Threading is often used in conjunction with asynchronous programming frameworks such as concurrent futures to perform non-blocking I/O operations. This allows you to initiate multiple I/O operations concurrently and continue executing other tasks while waiting for their completion.

6)Producer-Consumer Situations: Threading is commonly used in situations where multiple threads need to communicate or exchange data. For instance, in producer-consumer patterns, one thread produces data while another thread consumes it. Threading provides synchronization mechanisms such as locks, semaphores, or queues to ensure proper communication and coordination between threads.

7)Background Tasks: Threading can be used to perform background tasks while the main program continues its execution. This is useful for scenarios where you need to perform periodic tasks, maintenance operations, or data processing tasks in the background without blocking the main execution flow.

steps for creating simple thread in python :-

1)Import the required module: The Threading module is a built-in module in python that simplifies thread creation and management.

Syntax :- import threading

2)Define a function that represents the task you want to perform in a separate thread. This function will be executed concurrently.

Syntax :- def your_task(): # Code for the task you want to perform

3)Create a thread object, passing the function as the target and any arguments as needed.

Syntax :- thread = threading.Thread(target=your_task)

4) Start the thread by calling the start() method.

Syntax :- thread.start()

5)If needed, you can join the thread to wait for its completion using the join() method.

Syntax thread.join()

Example:-1)

```
import threading
```

```
def task():
```

```
    print("Execute task")
```

```
thread=threading.Thread(target=task)
```

```
thread.start()
```

```
thread.join()
```

```
print("main thread continues executing")
```

In this example, the your_task() function represents the task you want to perform in a separate thread. The threading.Thread() class is used to create a thread object, passing your_task as the target function. The start() method starts the thread, and the join() method is called to wait for the thread's completion. Finally, the main thread continues executing after the join.

Ex.2)Creating Thread by using thread class with implement run method only

```
import time
```

```
import threading
```

```
class A(threading.Thread):
```

```
    def run(self):
```

```
        print("thread1")
```

```
class B(threading.Thread):
```

```
def run(self):  
    print("thread2")  
  
obj1=A()  
obj2=B()  
obj1.start()  
time.sleep(2)  
obj2.start()
```

Methods of thread class :-

1)start() :-start a thread by calling start method

Ex.

```
import threading  
  
def show(a,b):  
    c=a+b  
    print(c)  
  
t=threading.Thread(target=show,args=(10,20))  
t.start()
```

2)Run() :- Entry point for a thread.

Ex.

```
import threading  
  
def show(a):  
    print("value of a in thread=",a)  
  
t=threading.Thread(target=show,args=(10,))  
t.run()
```

3) Sleep() :-suspend thread for a specified time.

Ex.1)

```
import time  
  
import threading  
  
def show(a):  
    print('value of an in thread1=',a)  
  
def display(a) :  
    print('value of an in thread2=',a)
```

```

t1=threading.Thread(target=show,args=(10,))
t2=threading.Thread(target=display,args=(20,))
t1.run()
time.sleep(5)
t2.run()

```

Ex. 2)

```

import threading

def show(a,b):
    print("value of a in thread=",a)
    print("value of b in thread=",b)

x=threading.Thread(target=show,args=(10,20))
y=threading.Thread(target=show,args=(30,40))
z=threading.Thread(target=show,args=(50,60))

x.run()
y.run()
z.run()

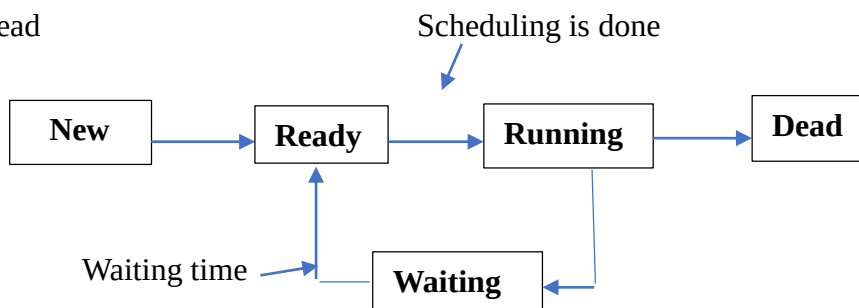
```

Thread life cycle:-

From thread creation to thread termination the thread goes different states are known as thread life cycle.

The life cycle has following step :-

- 1) New
- 2) Ready
- 3) Running
- 4)Waiting
- 5) Dead

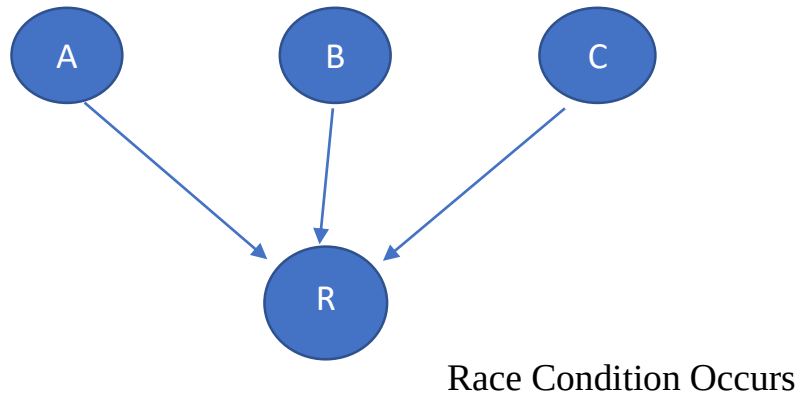


- 1) New :-** When we create the thread that is we create the object of that class that time it is in new state.
- 2) Ready :-** After creation of thread we call the start method on trade then it is in ready state.
- 3) Running :-** After scheduling is done it is sent to running state and in running state the actual execution is started.

4)Waiting :- During the execution of thread if any interruption is occurred i.e. if we call the sleep or join method on thread then it is waiting state.

5) Dead :-After successfully execution of thread is terminated i.e. the natural death of the thread it is in dead state.

Thread Synchronization :-



In this mechanism the more than one thread has allow to access the common data or common resources simultaneously, at that time there will be chance of race condition. The race condition we need to implement thread synchronization.

Synchronization :-it is a mechanism which allow only one thread at a time access the common data or common resource i.e. execute the critical section.

In python for the thread synchronization they have provided lock class, the lock class provides following two methods.

1)Acquire-thread must requires the lock by using acquire method before executing its critical section i.e. accessing the common data.

2)Release-the thread must need to release the lock by using release method after execution of its a critical section.

Ex.

1)

```
import threading
```

```
a=100
```

```
b=50
```

```
lock=threading.Lock()
```

```
def show():
```

```
    lock.acquire()
```

```
    c=a+b
```

```

print("Addition of thread1=",c)
lock.release()
def display():
    lock.acquire()
    c=a-b
    print("subtraction of thread2=",b)
    lock.release()
t1=threading.Thread(target=show)
t2=threading.Thread(target=display)
t1.start()
t2.start()

```

Exception Handling :

Exception handling is a technique to handle exception which maintains normal flow of a program.

Before understand exception handling we have to understand about exception.

Exception:

An exception is a condition which occurs when program encounters an abnormal error.(syntax error, runtime error, logical error).

When these exceptions occur, the Python interpreter stops the current process and passes it to the calling process until it is handled.

If these Exceptions not handled, the program will crash or stops immediately.

Exceptions are of two types:-

- a) Built-in Exceptions
- b) User defined Exception

a) Built-in Exceptions: The exceptions which are already defined by python are called as built-in exceptions.

Following table shows built-in exceptions

Sr.No.	Exception Name & Description
1	Exception
	Base class for all exceptions
2	StopIteration -Means repeating steps or instructions close single action performed.

	Raised when the next() method of an iterator does not point to any object.
3	SystemExit - Terminate the execution of program or end the application.
	Raised by the sys.exit() function.
4	StandardError
	Base class for all built-in exceptions except StopIteration and SystemExit.
5	ArithmeticError
	Base class for all errors that occur for numeric calculation.
6	OverflowError
	Raised when a calculation exceeds maximum limit for a numeric type.
7	FloatingPointError
	Raised when a floating point calculation fails.
8	ZeroDivisionError
	Raised when division or modulo by zero takes place for all numeric types.
9	AssertionError
	Raised in case of failure of the Assert statement.
10	AttributeError
	Raised in case of failure of attribute reference or assignment.
11	EOFError
	Raised when there is no input from either the raw_input() or input() function and the end of file is reached.
12	ImportError
	Raised when an import statement fails.
13	KeyboardInterrupt
	Raised when the user interrupts program execution, usually by pressing Ctrl+c.

14	LookupError
	Base class for all lookup errors.
15	IndexError
	Raised when an index is not found in a sequence.
16	KeyError
	Raised when the specified key is not found in the dictionary.
17	NameError
	Raised when an identifier is not found in the local or global namespace.
18	UnboundLocalError
	Raised when trying to access a local variable in a function or method but no value has been assigned to it.
19	EnvironmentError
	Base class for all exceptions that occur outside the Python environment.
20	IOError
	Raised when an input/ output operation fails, such as the print statement or the open() function when trying to open a file that does not exist.
21	IOError
	Raised for operating system-related errors.
22	SyntaxError
	Raised when there is an error in Python syntax.
23	IndentationError
	Raised when indentation is not specified properly.
24	SystemError
	Raised when the interpreter finds an internal problem, but when this error is encountered the Python interpreter does not exit.
25	SystemExit

	Raised when Python interpreter is quit by using the sys.exit() function. If not handled in the code, causes the interpreter to exit.
26	TypeError Raised when an operation or function is attempted that is invalid for the specified data type.
27	ValueError Raised when the built-in function for a data type has the valid type of arguments, but the arguments have invalid values specified.
28	RuntimeError Raised when a generated error does not fall into any category.
29	NotImplementedError Raised when an abstract method that needs to be implemented in an inherited class is not actually implemented.

b) User defined Exception : The exception defined by user is called as user defined exception.

Syntax to create user defined exception

```
raise exception
```

where raise- is a keyword which raises exception
exception-any user defined exception

Exception Handling : Exception handling is a **technique to handle exception which maintains normal flow of a program .**

For exception handling following blocks are used

1) try block/clause :-

In Python, exceptions can be handled using a try statement.

The critical operation which can raise an exception is placed inside the try block.

The try block diverts flow of program towards next block called **except block.**

Note: In single block **only one** try block will be used.

Syntax of try block

```
try:
    -----
    ---Logic---
```

2) except block/clause:-

The except block let us handle the exception by which our program don't get terminated .

For one try block we can use single or multiple except blocks.

Syntax of except block

```
except exceptiontype:
    body of except block
```

In above syntax **exceptiontype** is optional

3) finally block/clause :

The finally block lets us execute code, regardless of the result of the try- and except blocks. Finally block is an optional block.

Syntax of finally block

```
finally:  
    body of finally block
```

A simple program of exception handling([ZeroDivisionError](#))

```
import sys  
try:  
    a=10  
    b=0  
    c=a/b  
    print("Division is",c)  
except:  
    print("Can not divide by zero")  
finally:  
    print("finally block executed")
```

Output:

Can not divide by zero
finally block executed

In above program the code is enclosed in **try** block which raise built-in exception called [ZeroDivisionError](#) because we cannot divide by zero(a/b where a=10 , b=0).

The raised exception is diverted towards **except** block which will execute.

After that **finally** block gets executed.

Import sys:- This module contains methods & variables for modifying many of elements of python runtime environment.

Another example of exception handling (IndexError)

```
import sys  
try:  
    a=[10,20,30]  
    print("value of a=",a[5])  
except :  
    print("Invalid index number")
```

Output:

Invalid index number

In above program our code kept in try block from which we are trying to access index number a[5] Which is not available.

So the built-in exception called [IndexError](#) is raised and passed towards except block where the exception is handled

User defined Exception and User defined Exception Handling / Custom Exception :- Handling We already seen about user defined exception and how to raise an user defined exception from try block.

Now we are going to handle user defined exception in except block.

```
class myexception(Exception):  
    pass  
def show(age):  
    try:  
        if age<18:  
            raise myexception  
        else:  
            print("Allow")  
    except:  
        print("Not allow")  
show(10)
```

Python File Operation :-

File: File is a place on secondary storage where data can be **stored permanently**.

Since Random Access Memory (RAM) is volatile (which loses its data when the computer is turned off), we use files for future use of the data by permanently storing them.

In python Files are of two types ,

1) Text Files : Text file stores data in the **form of Characters**.

2) Binary files : Binary files stores data in the **form of bytes** i.e. group of 8 bits each

When we want to read from or write to a file(Both files Text and Binary), we need to open it first.

When we are done, it needs to be closed .

Hence, in Python, a file operation takes place in **the following order:-**

- 1) Open a file
- 2) Read or write (perform operation)
- 3) Close the file

1) Opening Files in Python :-

Python has a built-in open() function to open a file. This function returns a file object it is used to read, write or modify the file .

The **open()** function takes two parameters; *filename*, and file opening *mode*.

```
f = open("test.txt",'w') # write in text mode
```

We can specify the mode while opening a file. In mode, we specify whether we want to **read r**, **write w** or **append a** to the file. We can also specify if we want to open the file in text mode or binary mode. The default mode is reading in text mode. In this mode, we get strings when reading from the file. On the other hand, binary mode returns bytes and this is the mode to be used when dealing with non-text files like images or executable files.

File Opening Modes :-

Mode	Description
r	Opens a file for reading. (default)
w	Opens a file for writing. Creates a new file if it does not exist or truncates the file if it exists.
x	Opens a file for exclusive creation. If the file already exists, the operation fails.
a	Opens a file for appending at the end of the file without truncating it. Creates a new file if it does not exist.
t	Opens in text mode. (default)
b	Opens in binary mode.
+	Opens a file for updating (reading and writing)

2) **Read or write (perform operation)** : We will reading or writing text and binary files but before that we are going to understand closing operation

3) **Close the file** : After reading or writing a file we need to close the file as ,

```
f.close()
```

working with text files containing strings :-

Python File Methods :

There are various methods available with the file object. Here is the complete list of methods in text mode with a brief description:

Method	Description
close()	Closes an opened file. It has no effect if the file is already closed.
detach()	Separates the underlying binary buffer from the TextIOBase and returns it.
fileno()	Returns an integer number (file descriptor) of the file.

<code>flush()</code>	Flushes the write buffer of the file stream.
<code>isatty()</code>	Returns True if the file stream is interactive.
<code>read(n)</code>	Reads at most n characters from the file. Reads till end of file if it is negative or None.
<code>readable()</code>	Returns True if the file stream can be read from.
<code>readline(n=-1)</code>	Reads and returns one line from the file. Reads in at most n bytes if specified.
<code>readlines(n=-1)</code>	Reads and returns a list of lines from the file. Reads in at most n bytes/characters if specified.
<code>seek(offset,from=SEEK_SET)</code>	Changes the file position to offset bytes, in reference to from (start, current, end).
<code>seekable()</code>	Returns True if the file stream supports random access.
<code>tell()</code>	Returns the current file location.
<code>truncate(size=None)</code>	Resizes the file stream to size bytes. If size is not specified, resizes to current location.
<code>writable()</code>	Returns True if the file stream can be written to.
<code>write(s)</code>	Writes the string s to the file and returns the number of characters written.
<code>writelines(lines)</code>	Writes a list of lines to the file.

Now let's see Writing and reading operation on **Text** file first,

Text File Writing :

In order to write into a file in Python, we need to open it in write **w**, append **a** or exclusive creation **x** mode.

We need to be careful with the w mode, as it will overwrite into the file if it already exists. Due to this, all the previous data are erased.

Program to write data in Text file(w)

```
f=open('D:/test.txt','w')
f.write("Hi...")
f.close()
```

-In above program the file is opened by open() method. The open method contains two arguments first one is 'test.txt' (it is a file name which is stored in its default location i.e. python folder present in C drive) and second argument is 'w' file opening mode as write mode.

-After that we write a string **Hi** by using write() method. And after that we closed our file by close() method.

Text File Reading(r) : To read a file in Python, we must open the file in reading **r** mode.

There are various methods available for this purpose.

We can use the read(size) method to read in the size number of data.

If the size parameter is not specified, it reads and returns up to the end of the file.

```
f = open("D:/test.txt", "r")
m=f.read()
print(m)
f.close()
```

Reading Multiline Text File Line by Line:

we can read multiline text file line by Line for that **readlines()** method is used.

```
f = open("D:/test.txt", "r")
m=f.readlines()
for x in m:
    print(x)
f.close()
```

Text File Appending(a/a+) :-

In order to append a new line to the existing file, open the file in append mode, by using either 'a' or 'a+' as the access mode. The definition of these access modes are as follows:

-Append Only ('a'): Open the file **for writing**. The file is created if it does not exist. The handle is positioned at the end of the file. The data being written will be inserted at the end, after the existing data.

-Append and Read ('a+'): Open the file for **reading and writing**. The file is created if it does not exist. The handle is positioned at the end of the file. The data being written will be inserted at the end, after the existing data.

When the file is opened in append mode, the handle is positioned at the end of the file. The data being written will be inserted at the end, after the existing data. Let's see the below example to clarify the difference between write mode and append mode.

```
f=open('D:/test.txt','a')
f.write("How are You?")
f.close()
```

Output :Hi ...How are you

In above program we opened our previous file test.txt (which contain a string 'Hi') in append mode. After that the string **How are you** is append on file using write() function. The write function appends **How are you** at the end of string **Hi**.

Random Access in Binary Files :-

Binary File writing:

Before writing binary file first we have to open a file by same way but the file opening mode should be **wb** .

After that we can write a binary file by **write()** or **writelines()** method.

Program to write Binary file

```
f=open("D:/binfile.bin","wb")
num=[5, 10, 15, 20, 25]
arr=bytearray(num)    #converting to bytearray type
f.write(arr)
f.close()
```

Binary File reading:

Before reading binary file first we have to open a file with file opening mode **rb** .

After that we can read a binary file by **read()** or **readlines()** method.

```
f=open("D:/binfile.bin","rb")
p=f.read()                #reading binary file
num=list(p)               # conversion of binary data to list type
print (num)
f.close()
```

Binary File Modifying :- The r+b mode opens the file for both reading and writing in binary mode. The seek method is used to position the cursor, and the write method modifies the data at that position.

Ex.

Open the binary file for reading and writing

with open('d:/python/binfile.bin', 'r+b') as file:

Move the cursor to the 9th byte

file.seek(9)

Write new binary data at the current cursor position

file.write(b'awesome!')

seek() and tell() methods:

seek() method :

In Python, seek() function/method is used to **change the position of the File Handle** to a given specific position. File handle is like a cursor, which defines from where the data has to be read or written in the file.

Syntax:

```
f.seek(offset, from_what)
```

where **f** is **file pointer**

Parameters:

Offset: Number of positions to move forward

from_what: It defines point of reference.

The reference point is selected by the from_what argument. It accepts three values:

0: sets the reference point at the beginning of the file

1: sets the reference point at the current file position

2: sets the reference point at the end of the file

By default from_what argument is set to 0.

tell() method :

The tell() method returns the file handlers current position in a file stream.

Syntax

```
f.tell()
```

where f is file pointer

Example of seek() and tell() method :-

Note: test.txt file is already present in secondary storage in that file welcome(data) is present.

```
f=open("d:/test.txt",'r')
f.seek(3)
v=f.tell()
print('file pointer is on',v)
str=f.read()
print(str)
f.close()
```

Output: file pointer is on 3

come

Program of file with exception handling

```
try:
    f=open('test.txt','r')
    s=f.read()
    f.close()
    print(s)

except:
    print('can not perform operation on file')
```


working with directories(Python Directory and Files Management):

If there are a large number of files to handle in our Python program, we can arrange our code within different directories to make things more manageable.

A directory or folder is a collection of files and subdirectories. Python has the `os` module that provides us with many useful methods to work with directories (and files as well).

Get Current Directory :-

We can get the present working directory using the `getcwd()` method of the `os` module. This method returns the current working directory in the form of a string. We can also use the `getcwd()` method to get it as bytes object.

```
import os
os.getcwd()
```

Making a New Directory :-

We can make a new directory using the `mkdir()` method.

This method takes in the path of the new directory. If the full path is not specified, the new directory is created in the current working directory.

```
import os
os.mkdir('d:/test')
```

`os.listdir()` method in Python is used to get the list of all files and directories in the specified directory.

Renaming a Directory or a File :-

The `rename()` method can rename a directory or a file.

For renaming any directory or file, the `rename()` method takes in two basic arguments: the old name as the first argument and the new name as the second argument.

```
import os
os.rename('test','sangola')
```

Removing Directory or File :-

A file can be removed (deleted) using the `remove()` method. Similarly, the `rmdir()` method removes an empty directory.

Ex. 1) `import os`
 `os.rmdir('d:/test')`

Ex. 2) `import os`
 `os.remove('d:/abc/b.txt')`

Note: The `rmdir()` method can only remove empty directories.

In order **to remove a non-empty directory**, we can use the **rmtree()** method inside the **shutil** module.

Ex.

```
import shutil
shutil.rmtree('d:/abc')
```

with statement in Python :-

The with statement in Python is used for resource management and exception handling. It simplifies working with resources like files, network connections and database connections by ensuring they are properly acquired and released. When we open a file, we need to close it our self using close(). But if something goes wrong before closing, the file might stay open, causing issues. Using with open() automatically closes the file when we're done, even if an error happens.

example.txt file contains:

```
file=open("d:/example.txt","w")
file.write("Hello,World!")
f.close()
```

O/P- Hello, World!

Example 1 : Without with (Manual closing)

```
file = open("d:/example.txt", "r")
try:
    content = file.read()
    print(content)
finally:
    file.close() # Ensures the file is closed
```

Output :-

Hello, World!

Explanation: This code opens “example.txt” in read mode, reads its content, prints it and ensures the file is closed using a finally block.

Example 2: using with(Automatic closing)

```
with open("d:/example.txt", "r") as file:
    content = file.read()
    print(content) # File closes automatically
```

Output :-

Hello, World!

Explanation: with open(...) statement reads and prints the file's content while automatically closing it, ensuring efficient resource management without a finally block.

Using the with statement for file handling :-

File handling is one of the most common use cases for the with statement. When opening files using open(), the with statement ensures that the file is closed automatically after operations are completed.

Example 1 : Reading a file

```
with open("example.txt", "r") as file:
    contents = file.read()
    print(contents) # Print file content
```

Output:

Hello, World!

Explanation: Opens example.txt in read mode (“r”) and with ensures automatic file closure after reading and file.read() reads the entire file content into contents.

Example 2 : Writing to a file

```
with open("example.txt", "w") as file:
    file.write("Hello, Python with statement!")
```

Output:

Hello, Python with statement!

Explanation: The file is opened in write mode (“w”). After the with block, the file is automatically closed.

Working with binary files :-

Python Pickle Module :-

The pickle module is used for implementing binary protocols for serializing and de-serializing a Python object structure.

- **Pickling:** It is a process where a Python object hierarchy is converted into a byte stream.
- **Unpickling:** It is the inverse of the Pickling process where a byte stream is converted into an object hierarchy.

Module Interface

- **dumps()** – This function is called to serialize an object hierarchy.
- **loads()** – This function is called to de-serialize a data stream.

Constants provided by the pickle module

1. **pickle.HIGHEST_PROTOCOL**

This is an integer value representing the highest protocol version available. This is considered as the protocol value which is passed to the functions dump(), dumps().

2. **pickle.DEFAULT_PROTOCOL**

This is an integer value representing the default protocol used for pickling whose value may be less than the value of the highest protocol.

module provides the following four methods:

1. dump(): This method is used to serialize to an open file object
2. dumps(): This method is used for serializing to a string
3. load(): This method deserializes from an open-like object.
4. loads(): This does deserialization from a string.

Pickle a simple list: Pickle.py

```
import pickle
mylist = ['a', 'b', 'c', 'd']
with open('datafile.txt', 'wb') as fh:
    pickle.dump(mylist, fh)
print("Pickling completed!")
```

In the above code, list – “mylist” contains four elements (‘a’, ‘b’, ‘c’, ‘d’). We open the file in “wb” mode instead of “w” as all the operations are done using bytes in the current working directory. A new file named “datafile.txt” is created, which converts the mylist data in the byte stream.

Unpickle a simple list: unpickle.py

```
import pickle
pickle_off = open ("datafile.txt", "rb")
emp = pickle.load(pickle_off)
print(emp)
print("UnPickling completed!")
```

Output: On running above scripts, you can see your mylist data again as output. ['a', 'b', 'c', 'd']

Zippping Files :- ZIP file is a single file containing one or more compressed files.

or

ZIP file is a file format that is used for compressing multiple files together into a single file.

Why do we need zip files?

- 1.To reduce storage requirements.
- 2.To improve transfer speed over standard connections.

Zippping files in Python is a breeze with the zipfile module.

Example:

```
import zipfile
# Create a new zip file
with zipfile.ZipFile("example.zip", "w") as zfile:
    zfile.write("file1.txt")
    zfile.write("file2.txt")
print("Files zipped successfully!")
```

Output:Files zipped successfully!

Unzipping Files :-

Those files which are previously compressed or zipped that are uncompressed or unzipped is called as Unzipping files.

The Zip File class contains extractall() and extract() methods which are used for unzipping the files.

Unzipping files is as straightforward as zipping them. Let's unzip the files we zipped earlier:

Ex.

```
import zipfile
# Extract all contents of the zip file
with zipfile.ZipFile("example.zip", "r") as zfile:
    zfile.extractall("extracted_files")
print("Files unzipped successfully!")
```

Output:

Files unzipped successfully!

extractall() :-

The extractall() method is used to extract all the files present in the zip file to the current working directory. Files can also be extracted to a different location bypassing the path parameter.

Syntax: ZipFile.extractall(file_path , members=None, pwd=None)

Parameters:

- **file_path:** location where archive file needs to be extracted, if file_path is None then contents of zip file will be extracted to current working directory
- **members:** It specifies the list of files to be extracted, if not specified, all the files in the zip will be extracted. members must be a subset of the list returned by namelist()
- **pwd:** the password used for encrypted files, By default pwd is None.

Example 1: Extracting all the files present in the zip :-

Import the zipfile module Create a zip file object using ZipFile class. Call the extractall() method on zip file object and pass the path where the files needed to be extracted and Extracting the specific file present in the zip.

```
# importing the zipfile module
import zipfile
# Extract all contents of the zip file
with zipfile.ZipFile("example.zip", "r") as zfile:
    zfile.extractall("extracted_files")
print("Files extractall successfully!")
```

extract() :-

The extract() method is used to Extract a member from the zip to the current working directory. The file can also be extracted to a different location bypassing the path parameter.

Syntax: ZipFile.extract(member, file_path=None , pwd=None)

- **members:** It specifies the name of files to be extracted.
- **file_path:** location where archive file needs to be extracted, if file_path is None then contents of zip file will be extracted to the current working directory
- **pwd:** the password used for encrypted files, By default pwd is None.

Example 1: Extracting one files present in the zip :-

```
import zipfile
with zipfile.ZipFile("example.zip", "r") as zfile:
    zfile.extract("file1.txt",path="d:/zip/extract")
print("Files Extract successfully!")
```